

JAVACREAM

*Training
Consulting
Projectmanagement*

Python

- Name
- Rolle im Rahmen der Software-Entwicklung im Unternehmen
- Themenbezogene Vorkenntnisse, insbesondere Programmierung
- Aktuelle Problemstellung
- Konkrete individuelle Zielsetzung

Programmieren - Eine Einführung

- Datenerfassung
- Datenverarbeitungs-Pipeline
 - Exploration
 - Cleanup
 - Filtern
 - Transformieren
 - Aggregieren
- Ergebnis erstellen und speichern

- Datenerfassung
 - Datenverarbeitungs-Pipeline
 - Exploration
 - Cleanup
 - Filtern
 - Transformieren
 - Aggregieren
 - Ergebnis erstellen und speichern
 - Visualisierung der Ergebnisse, Diagramme und Reports
 - Story Tellings, "Von der Korrelation zur Kausalität"
- + Erkennen von Ausreißern / fehlenden Werten
 - + Deskriptive Statistik
 - + Regressionsanalyse
 - + Machine Learning



■ Automatisierung und Parametrisierung

- Datenerfassung
- Datenverarbeitungs-Pipeline
 - Exploration
 - Cleanup
 - Filtern
 - Transformieren
 - Aggregieren
- Ergebnis erstellen und speichern

- + Big Data
- + Ausführung
 - + lokal möglich,
 - + aber ohne jede Änderung auch in der Cloud

Ein abstrakter Programmablauf

- Datenerfassung
- Datenverarbeitungs-Pipeline
 - Exploration
 - Cleanup
 - Filtern
 - Transformieren
 - Aggregieren
- Ergebnis erstellen und speichern

+ Python kann einen
Programmlauf abbilden
+ Block 1 (3 Tage)
+ Block 2 (2 Tage)

Programmierung und Datenanalyse



- Datenerfassung
- Datenverarbeitungs-Pipeline
 - Exploration
 - Cleanup
 - Filtern
 - Transformieren
 - Aggregieren
- Ergebnis erstellen und speichern
- Visualisierung der Ergebnisse, Diagramme und Reports
- Story Tellings, "Von der Korrelation zur Kausalität"

+ Erkennen von Ausreißern / fehlenden Werten
+ Deskriptive Statistik
+ Regressionsanalyse
+ Machine Learning

+ Datenanalyse mit der
Pandas-Bibliothek
+ Block 3 (3 Tage)

Ein abstrakter Programmablauf

- Datenerfassung
- Datenverarbeitungs-Pipeline
 - Exploration
 - Cleanup
 - Filtern
 - Transformieren
 - Aggregieren
- Ergebnis erstellen und speichern

Python kann einen Programmablauf abbilden

Programmierung und Datenanalyse



- Datenerfassung
- Datenverarbeitungs-Pipeline
 - Exploration
 - Cleanup
 - Filtern
 - Transformieren
 - Aggregieren
- Ergebnis erstellen und speichern
- Visualisierung der Ergebnisse, Diagramme und Reports
- Story Tellings, "Von der Korrelation zur Kausalität"

- + Erkennen von Ausreißern / fehlenden Werten
- + Deskriptive Statistik
- + Regressionsanalyse
- + Machine Learning

+ Datenanalyse mit der Pandas-Bibliothek

Ein erstes Programmieren

- Aus dem Körpergewicht und der Körpergröße einer Person soll der "Body Mass Index" (ein Kriterium für Gesundheitsrisiken) bestimmt werden
- Formel zur Berechnung
 - Körpergewicht in kg
 - Körpergröße in Metern
 - Der BMI ist dann Körpergewicht geteilt durch Größe*Größe

Umsetzung mit Tabellenkalkulation

C1 : $\times$ $\checkmark$ f_x =A1/(B1*B1)					
	A	B	C	D	E
1	76,3	1,83	22,78360059		

 **bmi_calculator.py** ✕
You, 1 minute ago | 1 auth
1 **A1** = 76.3
2 **B1** = 1.83
3 **C1** = **A1**/**(B1*B1)**
4 **print**(**C1**)

Python Grundlagen der Programmierung

- Eine Variable ist ein Alias-Name auf einen Wert
 - Später (Block 2) noch etwas präziser formuliert
- Einer Variablen kann ein Wert zugewiesen werden
 - dies erfolgt mit dem =-Zeichen
- Werte werden formuliert mit sogenannten "Literalen"
 - 76.3
 - Gleitkommazahl
 - Trennzeichen ist immer der Punkt, kein Komma
 - 183
 - Ganzzahl
 - "Sawitzki", 'Rainer'
 - Zeichenketten
 - Doppelte und einfache Anführungszeichen sind gleichberechtigt
 - Hinweis: Sie müssen aber immer in Paaren auftreten ("Musterperson"))

- Nennen Sie Variablen sprechend
 - `A1 = 76.3`
 - `körpergewicht = 76.3`
- Variablennamen bitte englisch
 - `weight = 76.3`
- Beginnen immer mit Kleinbuchstaben
- Zur Betonung wird in Python meistens der Unterstrich genutzt
 - `bodymassindex` -> `body_mass_index`
 - Die Alternative ist die Verwendung von eingestreuten Großbuchstaben
 - `bodyMassIndex`
 - in Python eher unüblich

- Eine "Integrated Development Environment", eine IDE
 - Extension zur Python Programmierung
 - Dateiexplorer
 - Editor
 - Terminal / Konsole

- Eine Funktion muss aufgerufen werden
 - Angabe des Funktionsnamens
 - Ein paar runder Klammern
 - Optional: In den runden Klammern kann ein Wert oder eine Variable geschrieben werden
 - Dies ist ein sogenannter "Parameter"
- Ein Funktionsaufruf stößt eine Aktion an
 - Diese benutzt den übergebenen Parameter
- Beispiel
 - `print('Hugo')`
 - `print(body_mass_index)`

- print kann "alles" ausgeben
- print kann auch mehrere Parameter ausgeben
 - In den runden Klammern werden Parameter mit einem Komma getrennt
- Empfehlung
 - Für formatierte Ausgaben benutzen wir immer eine sogenannte "formatierte Zeichenkette"
 - Freitext, der mit Platzhaltern kombiniert werden kann
 - Platzhalter in Python ist ein paar geschweiffter Klammern
 - Literal der formatierten Zeichenkette ist f'...', f"..."
 - Nettes Feature: Formatierte Strings können

- Python verlangt eine strikte Syntax, die zu 100% erfüllt werden muss
 - Falls nicht: Fehler in der Ausführung
 - Python ist kein ChatGPT
- Fehler-Klassifikationen
 - Syntax Fehler
 - Verhindern die Programmausführung komplett
 - Laufzeitfehler
 - Das Programm wird in der Zeile des Laufzeitfehlers abgebrochen
 - Hinweis: Kontrolliertes Abbrechen, kein Reboot des Rechners notwendig
 - Logische Fehler
 - Falsch programmierte Algorithmen
 - Werden von Python nicht erkannt
 - ChatGPT schon
 - "Subtile Fehler" (Sawitzki-Begriff)
 - Noch nicht erklärbar, bei Auftreten bitte sofort Unterstützung meinerseits einfordern!

- Syntax Highlighting
- Code Vervollständigung
 - Vorsicht: Momentan viel zu ausführlich
- Umbenennen von Variablen
 - Einfaches Suchen und Ersetzen wie in allen Editoren
 - "Refactoring", hier "Rename Symbol" bzw F2
- Hilfefunktion und Dokumentation
 - Aktuell eher verwirrend
- Debugger
 - Debugger = "Entwanzer" -> Antike der IT mit Lochkarten...
 - Prinzip: Das Programm bleibt an einem Haltepunkt stehen und ermöglicht einen Blick in die Python-Anwendung
 - Sinn: Nachvollziehen eines komplexen Programmlaufs

- Benutzereingabe des Namens einer Person
- Revisited: Wertezuweisung an eine Variable
 - Auch das Ergebnis einer BuiltIn-Funktion kann einer Variablen zugeordnet werden
 - z.B. input
 - Wartet auf eine Benutzereingabe, die mit ENTER quittiert wird

■ Berechnungen = Mathematische Operatoren

- +
- -
- *
- /

Grundrechenarten

+ "Punkt vor Strich"

+ Runde Klammern zur Erhöhung der Übersicht

■ % -> Restzahldivision, Modulus

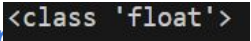
- 5%2 -> 1
- 4%2 -> 0
- 4%3 -> 1

■ **

- Potenzierung
- 3**2 -> 3 * 3 = 9
- 3**3 -> 3*3*3 = 27

Vollständige Liste in der Python-Dokumentation (aktuell nicht empfehlenswert, weil zu technisch) oder z.B. im Python-Tutorial der w3-school

+ https://www.w3schools.com/python/python_operators.asp

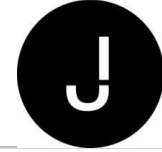
- Benutzereingabe von Gewicht und Größe
- Neu: Typisierung
 - Ein Wert in Python hat einen Typen
 - Der Typ kann mit der BuiltIn-Funktion "type" bestimmt werden
 - Ausgabe:

 - Bisherige Typen sind
 - str = String = Zeichenkette
 - int = Ganzzahl
 - float = Kommazahl
 - Später noch weitere Typen wie bool, list, ...
 - Anhand des Typen werden die zulässigen Operationen bestimmt
 - Zur Umwandlung von einem Typen in einen anderen gibt es eigene BuiltIn-Funktionen
 - Name = Name des Ziel-Typen
 - Versuche unzulässiger Umwandlungen führt zu einem Laufzeitfehler



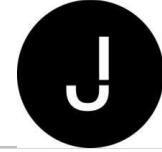
- `print`
 - Konsolenausgabe
 - `print('Hello')`
- `input`
 - Eingabeaufforderung
 - `value = input(prompt)`
- `type`
 - Bestimmung des Datentyps
 - `type_of_param = type(param)`

- Neue BuiltIn-Funktionen zur Typumwandlung
 - str
 - int
 - float
- Arbeitsweise
 - `value_as_string = str(value)`
 - funktioniert immer
 - `value_as_int = int(value)`
 - das kann einen Laufzeitfehler verursachen
 - dieser führt zu einem Programmabbruch, später werden wir dies anders lösen
 - `value_as_float = float(value)`
 - das kann einen Laufzeitfehler verursachen
 - dieser führt zu einem Programmabbruch, später werden wir dies anders lösen

- Eine Variable hat in Python keinen festen Typen
 - `str name = 'Hello'` ist in Python nicht möglich
 - oder: Python ist eine dynamisch typisierte Sprache
- Eine erneute Wertzuweisung an eine Variable ist immer möglich
 - `value = 'Hugo'`
 - ...
 - `value = 42`
 - ...
 - `value = 47.11`
 - `value = value + 1`
 - `value = int(value)`

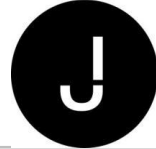


- https://www.w3schools.com/python/gloss_python_comparison_operators.asp
-



- < 18.5
 - Untergewicht
- $18.5-22$
 - Normalgewicht
- $22-30$
 - Übergewicht
- > 30
 - Fettleibig

- In Python ist das eine if-elif-else-Konstruktion
 - Arbeitet mit logischen Werten



- Neuer Datentyp
 - bool
- Literale
 - True
 - False

- Vergleichsoperatoren

- ==

- Gleichheit
 - Vorsicht: Ein einzelnes = ist immer eine Zuweisung

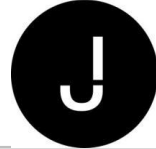
- !=

- Ungleichheit,

- Größer, kleiner, ...

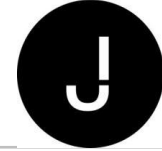
- >
 - <
 - >=
 - <=

Funktionieren mit Zahlen



- Mathematische Grundlagen von George Boole
- Negation
 - not
- Und-Verknüpfung
 - and
 - Ergebnis ist nur dann True, wenn beide Operanden True sind, sonst immer False
- Oder-Verknüpfung
 - or
 - Ergebnis ist nur dann False, wenn beide Operanden False sind, sonst immer True

- Allgemein
 - Der Ablauf des Programms wird in einzelne getrennte Bereiche ("Blöcke") unterteilt / strukturiert
 - Zur Definition eines Blocks verwenden die allermeisten Programmiersprachen geschweifte Klammern
 - In Python wird die Struktur durch einheitliche Einrückung definiert



- Bisher
 - if *logischer Wert*:
 - falls wahr...
 - else:
 - falls falsch
- Neu
 - if *logischer Wert*:
 - falls wahr...
 - elif *anderer Wert*:
 - else:
 - falls falsch

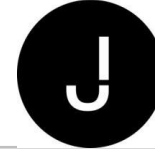
- Umsetzung der BMI-Wertung von Slide 28
- Ausgabe soll sein
 - name hat mit einem Gewicht von ... und einer Körpergröße von ... einen BMI von ... und ist damit ...

- Vor der Berechnung des BMI sollen Körpergröße und Gewicht validiert werden
 - Regel Gewicht
 - $> 2, < 450$
 - Regel Größe
 - $> 35, < 275$
 - Fehleingaben sollen statt der bisherigen Ausgabe durch eine Ausgabe der Form "Fehlerhafte Eingabe: ..." ersetzt werden
- Nach der Berechnung des BMI soll dieser auf Plausibilität geprüft werden
 - Falls ein BMI kleiner 14 oder größer als 45 berechnet wird, wird die Ausgabe auf "unzulässiger BMI..." geändert

- Unser Programm zum Aufwärmen:
 - Umgebung vorbereiten
 - Remote Rechner öffnen
 - GitHub
 - Digitales Flipchart
 - Übungen auf den aktuellen Stand = Lauffähige Versionen bringen
 - Gibt es Fragen / Wünsche / Anmerkungen?
- Das Programm für den heutigen Tag umfasst Schleifen und statische
= unveränderbare Collections

- Wiederholte Ausführung eines Blocks von Anweisungen
- Beispiel
 - Die Eingabe von Name, Gewicht, Größe soll nach Ausgabe wiederholt werden
- while-Schleife
 - Diese prüft eine Bedingung
 - Solange diese erfüllt ist, wird der Block der while-Schleife wiederholt
 - Vorsicht
 - Es gibt "Endlosschleifen", die den Rechner stark belasten können

- Nach Eingabe der Werte erfolgt die BMI-Berechnung
- Danach Rückfrage: Nochmal? j|n
- Wenn j -> Wiederholung der Eingabe
- Wenn n -> Programm beenden
- Hinweise
 - Ich empfehle hierfür ein while True mit einem break
 - == funktioniert auch mit Zeichenketten
 - if again == 'j':
 - Bitte niemals Variablen so wie BuiltIn-Funktionen nennen!
 - input = input("Nochmal: j|n")
- Ergänzend
 - Schleife von bis, Grenzen über Benutzereingabe



Datencontainern = "Collections"

- Der Begriff Collections ist sehr allgemein und wird faktisch von den allermeisten Programmiersprachen unterstützt
- In der realen Welt sind solche Dinge weit verbreitet
 - Einkaufsliste, Aktenordner, Telefonbuch, Wörterbuch
- Collections sind auch ein akademisch mathematisches Konzept
 - Intervalle (=Wertebereiche), Mengen mit Schnittmenge und Vereinigungsmenge

- Eventuell auch als "Array" bekannt
 - Analogie: Zeile oder Spalte einer Tabelle
- Eine Liste in Python wird über ein Literal definiert
 - [Kommaseparierte Aufzählung der Elemente]
- Jede Liste hat eine Länge
 - Bestimmbar durch die neue BuiltIn-Funktion len
- Elemente einer Liste sind über einen fortlaufenden Index nummeriert
- Neue Syntax, um eine Aktion anzustoßen
 - Bisher: Runde Klammern mit Parameter stoßen eine BuiltIn-Funktion an
 - Nun: Eckige Klammern mit Indexwert greifen auf das Element zu
- Iteration über die Liste in den meisten Fällen mit einer for in -Schleife

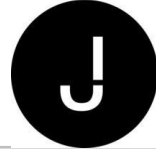
- Die Eingangsdaten für die BMI-Berechnung sollen aus einer Liste genommen werden
 - Aufbau: [['Hugo', 76.6, 185], [...]]
- Hinweis
 - Die Liste wird als Variable im Programm eingeführt
 - später = heute oder Morgen werden wir stattdessen eine Datei auslesen
 - Ein dynamisches Erzeugen der Liste ist NICHT vorgesehen
 - später = Morgen
- Als Ergebnis soll für jeden Listeneintrag auf der Konsole die Kategorisierung stehen

- Ein Tuplel in Python wird über ein Literal definiert
 - (Kommaseparierte Aufzählung der Elemente)
- Jedes Tuplel hat eine Länge
 - Bestimmbar durch die neue BuiltIn-Funktion len
- Elemente eines Tupels sind über einen fortlaufenden Index nummeriert
- Neue Syntax, um eine Aktion anzustoßen
 - Bisher: Runde Klammern mit Parameter stoßen eine BuiltIn-Funktion an
 - Nun: Eckige Klammern mit Indexwert greifen auf das Element zu
- Iteration über das Tuplel in den meisten Fällen mit einer for in

Der Unterschied zu einer Liste wird erst sichtbar, wenn wir dynamisch Elemente zu verwalten

- + List ist dynamisch
- + Tuplel ist unveränderbar

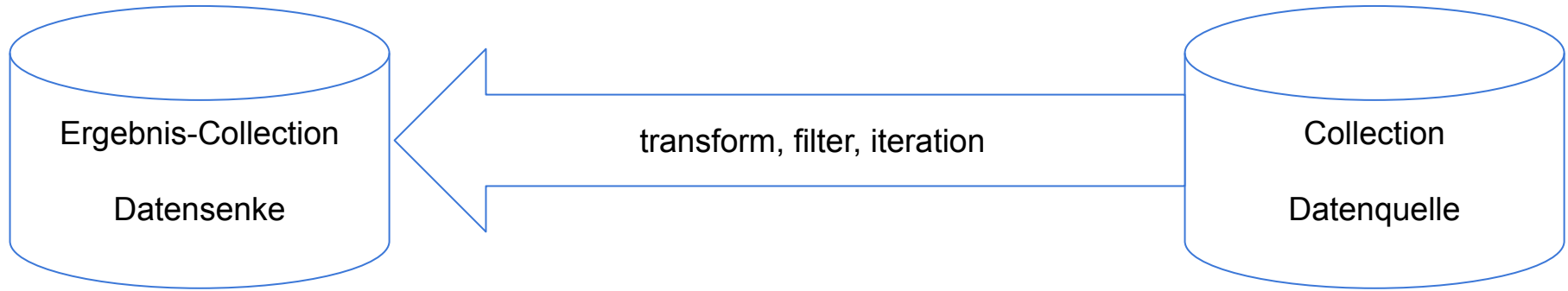
- Ist mathematisch ein Intervall
 - Von ... bis von Zahlen
- Für Ranges gibt es kein Literal
 - "Wir haben nicht genügend Symbole"
 - Zur Erzeugung wird die BuiltIn-Funktion range genutzt
 - Hinweis: Es gibt auch list() erzeugt eine leere Liste, dict() ein leeres Dictionary



- In der Mathematik als "Menge" bezeichnet
- Ein Set hält eine Menge von Elementen, aber ohne "Duplikate" #
- Ein Set hat keine innere Ordnung
 - Damit ist ein Index-Zugriff nicht möglich

- Auch Strings sind als Folge von Einzelzeichen "Datencontainer"
- Damit
 - len
 - Indexzugriff
 - for in

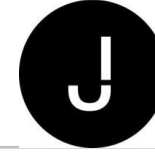
- Für Collections kann die folgende Abfrage formuliert werden
 - if element **in** collection:
- "Slicing"
 - Von bis wie beim range
 - Ergebnis ist eine Subliste
 - Ein **Slice** wird im Indexzugriff formuliert
 - Syntax
 - numbers[0] -> Das nullte Element
 - numbers[0:2]
 - numbers[1:4:2]
 -



- Ein "Wörterbuch"
 - Suche eines Wertes (=englisches Wort) anhand eines Schlüssels (=deutsches Wort)
- Ein Dictionary besteht aus Key-Value-Paaren
- Dictionaries haben eine Länge
- Zugriff auf die Values erfolgt über den Key
 - Wie bei der Liste werden eckige Klammern benutzt
 - Ein Indexzugriff ist nicht zulässig, ein Dictionary hat keine innere Ordnung
- Keys sind im Dictionary immer eindeutig
 - Unterschied zur Liste, eine Liste kann einen Wert natürlich mehrfach enthalten

- Ausgangssituation: `postal_codes.py` enthält dictionaries für die Suche einer Stadt nach bekannter PLZ bzw. die PLZs einer bekannten Stadt
- Schreiben Sie eine Anwendung, in der die benutzende Person eine gültige PLZ eingibt, das Programm gibt die gefundene Stadt aus
 - Hinweis
 - Unbekannte Keys führen zu einem Programmfehler
 - ist OK...

- Das Programm für den heutigen Tag umfasst Objekte und Methoden
 - damit auch dynamische Collections
- Und dann schreiben wir eigene Funktionen!



Die Python Bibliothek

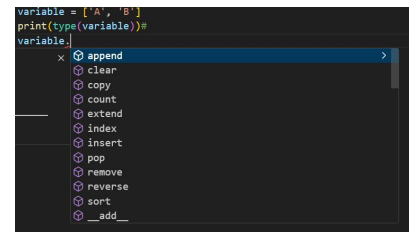
- Eine Sammlung von ausführbaren Aktionen
 - Flexibel
 - Bibliotheken können eingebunden werden
- Im Gegensatz dazu ist die Syntax einer Programmiersprache "fest"
 - Änderungen in der Syntax können nur durch einen neuen Release erfolgen

- Operatoren und Schlüsselwörter von Python
- BuiltIn-Funktionen der Python-Standard-Bibliothek
 - Diese Standard-Bibliothek ist Bestandteil der Python-Installation
- Exkurs
 - Unterschied Operatoren - BuiltIn-Funktion
 - Python
 - +-Operator
 - $n1 + n2$
 - print()-Built-In-Funktion
 - Alternative, hypothetische Sprache
 - plus-BuiltIn-Funktion
 - `plus(n1, n2)`
 - -Operator
 -  "Hello"

- Dokumentation
 - Python-Dokumentation
 - <https://docs.python.org/3/library/functions.html>
 - W3 School
 - https://www.w3schools.com/python/python_ref_functions.asp
- Wertung
 - Wir haben etwa 60 BuiltIn-Funktionen
 - Davon kennen wir bereits etwa 10
 - Weitere 10 sind einfach erschließbar (sum, min, max, ...)
 - Der Rest ist immer sehr technisch und damit für unser Seminar nicht nötig

- Bisher
 - Eine Variable ist ein Alias-Name auf einen Wert
 - Der Wert wird durch ein Literal definiert
 - ist das Ergebnis einer Operation
 - ist das Ergebnis eines Aufrufs einer BuiltIn-Funktion
- Nun
 - Eine Variable ist ein Alias-Name eines **Objekts**
 - Ein Objekt ist etwas, das Eigenschaften **hat** und Funktionalitäten **kann**
 - In der Programmierung hat ein Objekt **Attribute** (=Eigenschaften) und **Methoden** (=Funktionalitäten)

- Nutzen bereits permanent Variablen und damit Objekte
- Die Attribute und Methoden haben wir aber bisher nicht gesehen und damit und damit auch nicht genutzt
- Um eine Übersicht der vorhandenen Attribute und Methoden zu bekommen wird der Typ eines Objekts benötigt
 - In Python: type-BuiltIn-Funktion
 - Dokumentation
 - <https://docs.python.org/3/tutorial/datastructures.html>
 - https://www.w3schools.com/python/python_lists_methods.asp
 - Code Assist von Visual Studio Code



```
variable = ['A', 'B']  
print(type(variable))  
variable.  
x  append  
  clear  
  copy  
  count  
  extend  
  index  
  insert  
  pop  
  remove  
  reverse  
  sort  
  __add__
```

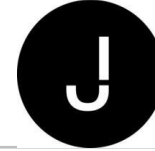
- z.B. in der counting.py ergänzen:
 - bei Fehleingabe (Eingabe kann nicht in ein int umgewandelt werden)
 - Ausgabe einer Meldung: Kann nicht als Zahl interpretiert werden
 - Abfrage: Nochmal versuchen?
- Neues Programm
 - Definiert eine Zeichenkette, z.B. 'Das ist ein langer Text mit verschiedenen Wörtern'
 - Analyse dieses Textes
 - Aus wievielen Buchstaben besteht dieser Text?
 - Mit und ohne Leerzeichen
 - Wie viele 'a's sind in diesem Text?
 - Wie viele Vokale sind in diesem Text (Vokale also a, e, i, o, u)?
 - Aus wievielen Wörtern besteht dieser Text?

- Einlesen der Datei people.txt
- Erstellen einer Resultatliste, die alle Namen in GROSSBUCHSTABEN umgewandelt enthält
- Dieses Ergebnis dann in eine Datei people_upper.txt speichern
- Erweitern Sie die Personenliste um weitere Namen und implementieren Folgendes
 - Unique Names -> unique_people.txt
 - Resultat in der Form {name} has {n} characters
 - ...

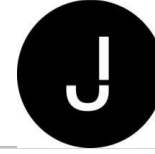
- List
 - Verwendung als dynamische Liste mit beliebiger Anzahl von Elementen
- Dictionary
 - **Suche** eines Wertes an Hand eines Schlüssels
- Range
 - Eine Aufzählung, häufig verwendet in einer Schleife
- Set
 - Verwendung als dynamische Liste mit beliebiger Anzahl von Elementen ohne **Duplikate**
- Neu
 - Tupel
 - Verwendung als Liste mit beliebiger Anzahl von **unveränderbaren** Elementen

- Bisher
 - Rein lesend
 - `[index]`
 - `[index-slice]`
- Neu
 - Schreibend
 - `my_list[index] = value`
 - `my_list[index-slice] = value`

Sinnvoll für list und dict



- `vowels_counter`
 - Statt set der Vokale ein Tupel
 - Mehrwert: Garantierte Unveränderlichkeit
 - Prüfen auf Vokale nur mit Klein- (Großbuchstaben)
 - Umwandlung des Einzelbuchstabens in den Kleinbuchstaben
 - Zählen der Vokale könnte der String selber machen
- `bmi-Beispiel`
 - Kategorisierung durch Verwendung eines Dictionaries
 - Ausgabe durch Iteration über das Dictionary
- `Postleitzahlen`
 - Auslagern der PLZs in externe Datei



Funktionen

- Eine Funktion hat einen Namen
- Eine Funktion stellt Aktionen zur Verfügung, die beim Aufruf ausgeführt werden
 - das ist die Implementierung der Funktion = eine Sequenz von Anweisungen
- Der Aufruf einer Funktion erfolgt durch ein rundes Klammernpaar
- Die Aktionen bekommen für ihre interne Verarbeitung Daten "von außen" = vom Aufrufer über Parameter
- Das Ergebnis einer Aktion kann einer Variable zugeordnet werden

- BuiltIn-Funktionen

- print

- Name ist 'print'
 - print hat eine beliebig lange Liste von Parametern
 - Implementierung macht "nur" eine Konsolenausgabe
 - die allerdings technisch sehr aufwändig ist
 - print liefert kein Ergebnis zurück

- sum

- Name ist 'sum'
 - Parameter muss eine Collection sein
 - Implementierung iteriert über die Collection und summiert alle Elemente mit dem Plus-Operator
 - Rückgabewert ist die Summe

- Methoden = Funktionen, die einem Objekt zugeordnet sind
 - Aktionen einer Methode beziehen sich immer auf das Objekt
- `numbers.append`
 - Name: 'append'
 - Parameter ist das hinzuzufügende neue Element
 - Intern wird diese Liste um ein Element vergrößert
 - Kein Rückgabewert
- Hinweis
 - Es gibt keine `append-BuiltIn-Funktion`, aber wenn es sie gäbe hätte sie folgende Parameter
 - `numbers = [1, 3]`
 - `built_in_append (numbers, 42)`

- Pseudo-Umsetzung

```
my_sum p1, p2:
```

```
    # mache etwas mit diesen Parametern
```

```
    result = p1 + p2
```

```
sum_of_numbers = my_sum(47, 11)
```

- Python-Umsetzung

```
def my_sum (p1, p2):
```

```
    # mache etwas mit diesen Parametern
```

```
    result = p1 + p2
```

```
    return result
```

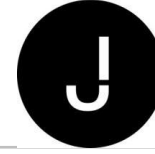
```
sum_of_numbers = my_sum(47, 11)
```

- Bisher
 - "Auf in die Tasten und mal schauen was rauskommt"
 - Qualitätskriterium ist "Anwendung funktioniert"
- Nun
 - Anforderungen
 - Umgesetzt in einen Ablaufplan
 - Ablaufplan wird in Python umgesetzt
 - Dafür werden Funktionen genutzt
 - Dieser Ablaufplan wird während der Programmierung mit Programmanweisungen befüllt ("implementiert")
 - bei Bedarf der Plan auch erweitert

- Eine Textdatei soll eingelesen und analysiert werden
- Ablaufplan
 - Datei einlesen
 - Anzahl Zeichen
 - Alphanumerische Zeichen
 - Anzahl Wörter
 - Anzahl unterschiedlicher Wörter
 - Größte und kleinste Wortlänge
 - Ausgabe der Ergebnisse

- `people_data.txt`
 - Firstname,Lastname,Weight,Height
- Daraus soll eine Ergebnis-Datei erzeugt werden, in der steht
 - Firstname Lastname hat mit einem Gewicht von... und einer Größe von ... einen BMI von... und ist damit ...
- Grober Ablaufplan
 - `get_data()`
 - `get_people_data()`
 - Pro Zeile müssen die Personen-Informationen extrahiert werden
 - Hinweis: split-Methode des str, Trennzeichen ist ','
 - `get_people_result()`
 - `write_people_result()`
- Hinweis
 - Intern können selbstverständlich noch weitere Funktionen sinnvoll sein

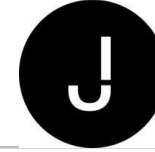
- list, tuple, dict, set, range, str
 - range -> Aufzählung von Zahlen
 - str -> Zeichenkette
- Indexzugriff = Innere Ordnung = Zeilenweise Anordnung
 - list Literal: [1,2,3,2]
 - Die Liste selber kann jederzeit neue Elemente hinzugefügt bekommen, Elemente löschen, clear
 - tuple Literal (1,2,3,2)
 - unveränderlich
- Über einen Key wird ein Value bestimmt = Suchalgorithmus bzw. eine Gruppierung
 - dict Literal {key1: value1, key2: value2}
- Menge (= keine innere Ordnung) ohne Duplikate
 - set Literal {1,2,3,2}
 - dieses Set hat nur drei Elemente, 1,2,3



Module in Python

- Ein Modul ist ein ganz normales Python-Programm
 - damit haben wir bereits immer mit Modulen gearbeitet (!)
 - Im Detail sind Module doch etwas mehr, aber das lassen wir im Lernpfad weg
- Jedes Modul hat einen Namen
 - Dieser Name wird aus dem Dateinamen abgeleitet
 - Endung .py wird weggelassen
- Alles, was in diesem Python-Programm definiert wird, ist Bestandteil des Moduls
 - Vorsicht: Module sollen beim Einladen keine Anweisungen ausführen

- Erfolgt durch die Anweisung `import`
 - `import modul_name`
- Durch den Import werden die Funktionen und Variablen eines Moduls einem Objekt zugeordnet, das den Namen des Moduls hat
 - `import modul_name as alias`

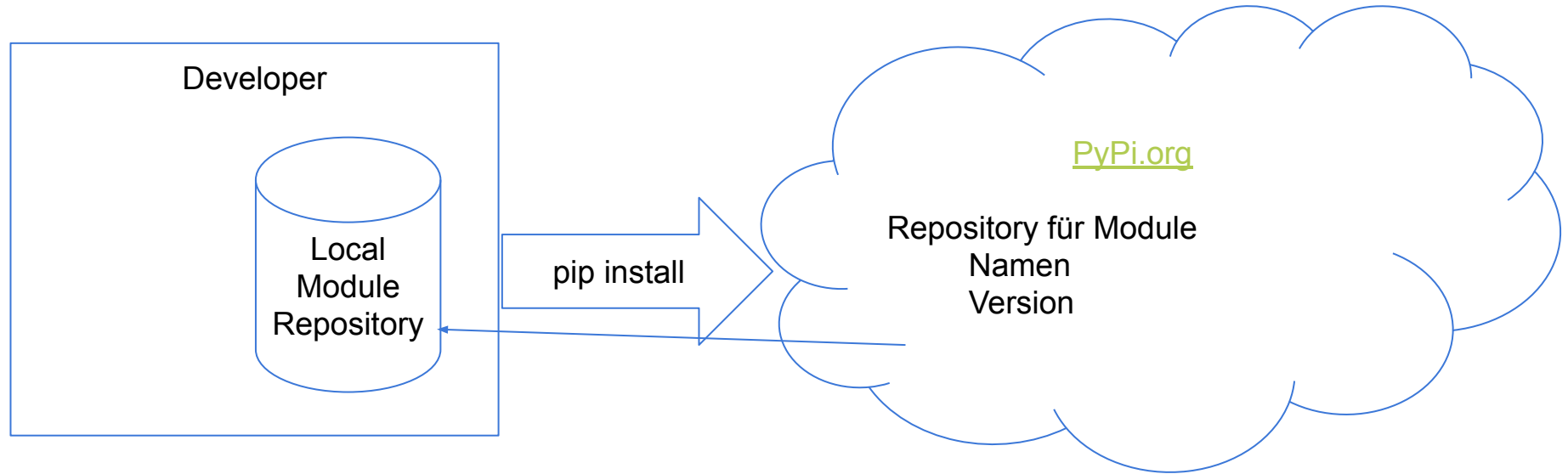


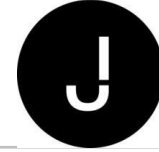
- Nachvollziehen shopping_list
- Transfer auf das text_analysis-Beispiel



- Mathematische Berechnungen
 - Modul math
- Datum und Uhrzeit
 - Modul datetime
- Zufallszahlengeneratoren
 - random
- ...

- Erzeugt kein Objekt
- Die Funktionen werden wie BuiltIn-Funktionen angesprochen
 - Vorsicht: Das kann zu Mehrdeutigkeiten führen
- `from modul_name import one_function`
- `from modul_name import function1, function2`
- `from module_name import *`

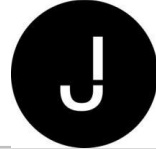




- importieren von Standard-Module
- math- und datetime-Modul
 - w3school-Dokumentation
 - Python-Dokumentation

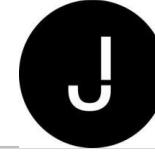
Exkurs: Informationen laden aus dem Internet

- Informationen im Internet werden über eine URL bestimmt
- Ergebnistypen
 - HTML = Formatiertes Textdokument
 - Nutzung in Python-Programmen eher nicht
 - Mediendateien
 - Nutzung in Python-Programmen eher nicht
 - Pure Daten im JSON-Format
 - JSON ist de facto Standard
 - Im Prinzip eine Kombination von Dictionary und List, allerdings nicht ganz Python
 - Beispiel: JSON `true`, Python `True`



- Dafür nutzen wir das requests-Modul
 - Eventuell installieren mit pip install requests
- Für den lesenden Zugriff gibt es eine Methode namens get(url)
 - Hinweis: Auch Schreiboperationen könnten aufgerufen, post, put, delete

- Funktionalität "Buch Suche" auf javacream.eu
 - Parameter: Ein ISBN-String
 - vorhanden sind ISBN1, ISBN2, ISBN3, ISBN4
 - Rückgabewert ist ein Dictionary mit isbn, title, price, available
 - Implementierung geht auf die URL
 - <http://javacream.eu:8080/api/books>
 - Eine Liste der Buchinformationen
 - <http://javacream.eu:8080/api/books/ISBN1>
 - Das Buch mit der isbn ISBN1
 - http://javacream.eu:8080/api/books/ISBN_NONEXISTENT
 - Ein leeres Dictionary
- Eine Anwendung mit der Eingabe der ISBN und der Konsolenausgabe des Titels und des Preises



Objektorientierte Programmierung

Was ist ein Book?

Book

Article Talk
From Wikipedia, the free encyclopedia

For other uses, see *Book (disambiguation)*.

A **book** is a structured presentation of recorded *information*, primarily verbal and graphical, through a medium. Originally physical, *electronic books* and *audiobooks* are now existent. Physical books are objects that contain printed material, mostly of writing and images. Modern books are typically composed of many pages *bound* together and protected by a *cover*, what is known as the *codex* format; older formats include the *scroll* and the *tablet*.

As a *conceptual object*, a *book* often refers to a *written* work of substantial length by one or more *authors*, which may also be distributed digitally as an electronic book (*ebook*). These kinds of works can be broadly *classified* into *fiction* (containing invented content, often narratives) and *non-fiction* (containing content intended as factual truth). But a physical book may not contain a written work: for example, it may contain *only* drawings, engravings, photographs, sheet music, *puzzles*, or removable content like *paper dolls*.

The modern book industry has seen several major changes due to new technologies, including ebooks and audiobooks (recordings of books being read aloud). Awareness of the needs of *print-disabled* people has led to a rise in *formats designed for greater accessibility* such as *braille* printing and *large-print* editions.

Google Books estimated in 2010 that approximately 130 million total unique books had been *published*. The book publishing process is the series of steps involved in book creation and dissemination. *Books are sold* at both regular stores and specialized bookstores, as well as online (for delivery), and can be borrowed from *libraries* or *public bookcases*. The *reception* of books has led to a number of social consequences,



The Gutenberg Bible, published in the mid-15th century, one of the first books to be printed using the printing press



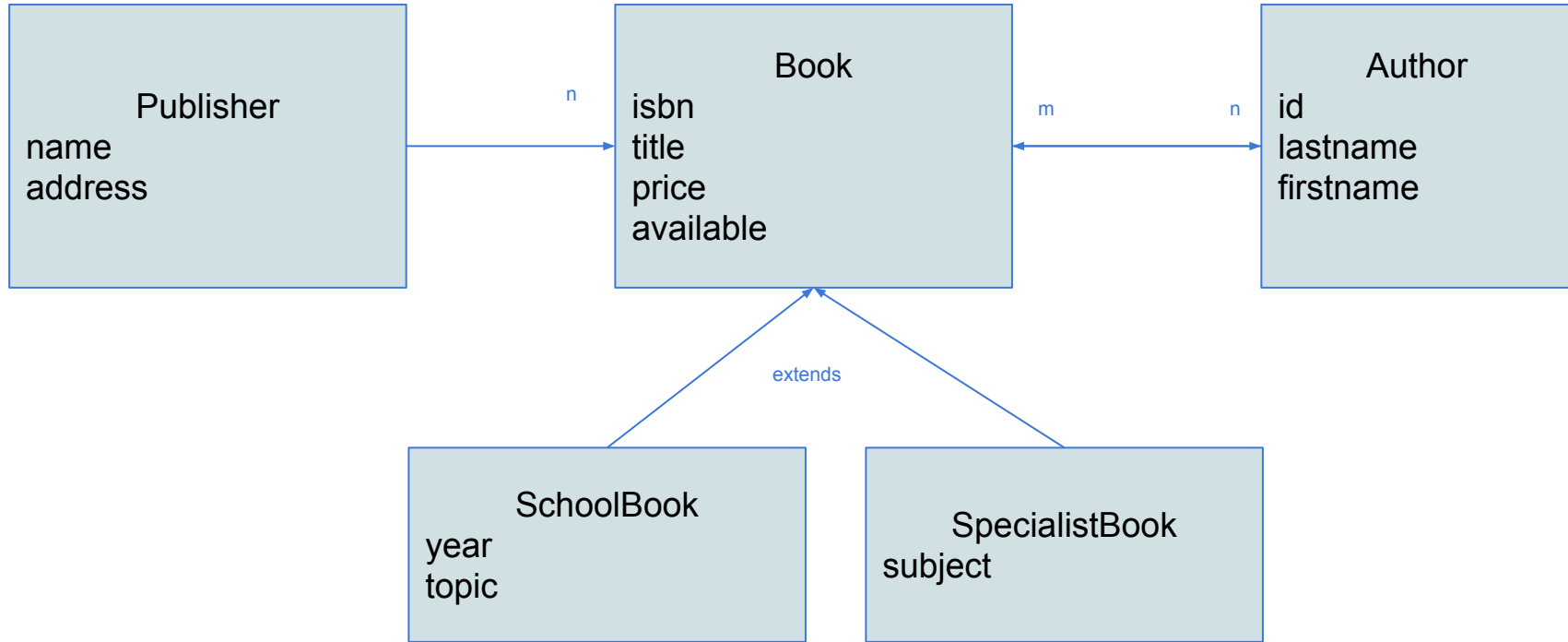
Abstraktion

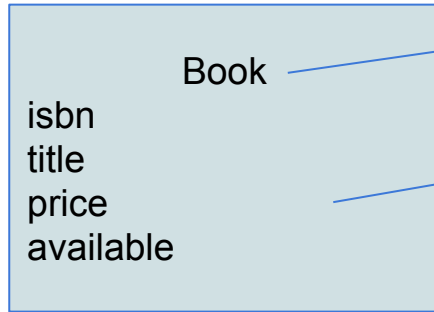
Book

isbn
title
price
available

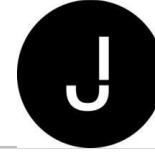
Das ist ein einfaches aber trotzdem
vollständiges "Klassendiagramm"
eines Modells

Exkurs: Ein komplexeres Klassendiagramm



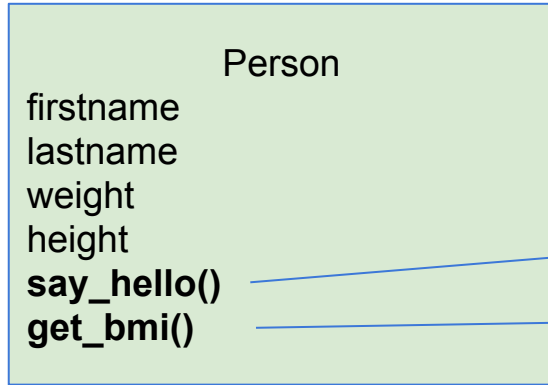


```
class Book:
    def __init__(self, isbn, title, price, available):
        self.isbn = isbn
        self.title = title
        self.price = price
        self.available = available
```



Person

firstname
lastname
weight
height



```

class Person:
    def __init__(self, firstname, lastname, weight, height):
        self.firstname = firstname
        self.lastname = lastname
        self.weight = weight
        self.height = height
    def say_hello(self):
        return f'Hallo, mein Name ist {self.firstname} {self.lastname}'
    def get_bmi(self):
        return self.weight / (self.height ** 2)
    
```

Vollständige Klasse inklusive `__repr__`

```
class Person:
    def __init__(self, firstname, lastname, weight, height):
        self.firstname = firstname
        self.lastname = lastname
        self.weight = weight
        self.height = height
    def __repr__(self):
        return f'Person(firstname={self.firstname}, lastname={self.lastname}, weight={self.weight}, height={self.height})'
    def say_hello(self):
        return f'Hallo, mein Name ist {self.firstname} {self.lastname}'
    def get_bmi(self):
        return self.weight/(self.height ** 2)
```

Address

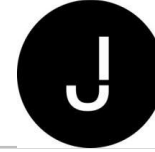
city
street
postal_code

Customer

id
name
limit

ShoppingItem

product_name
amount



Datenbankzugriff

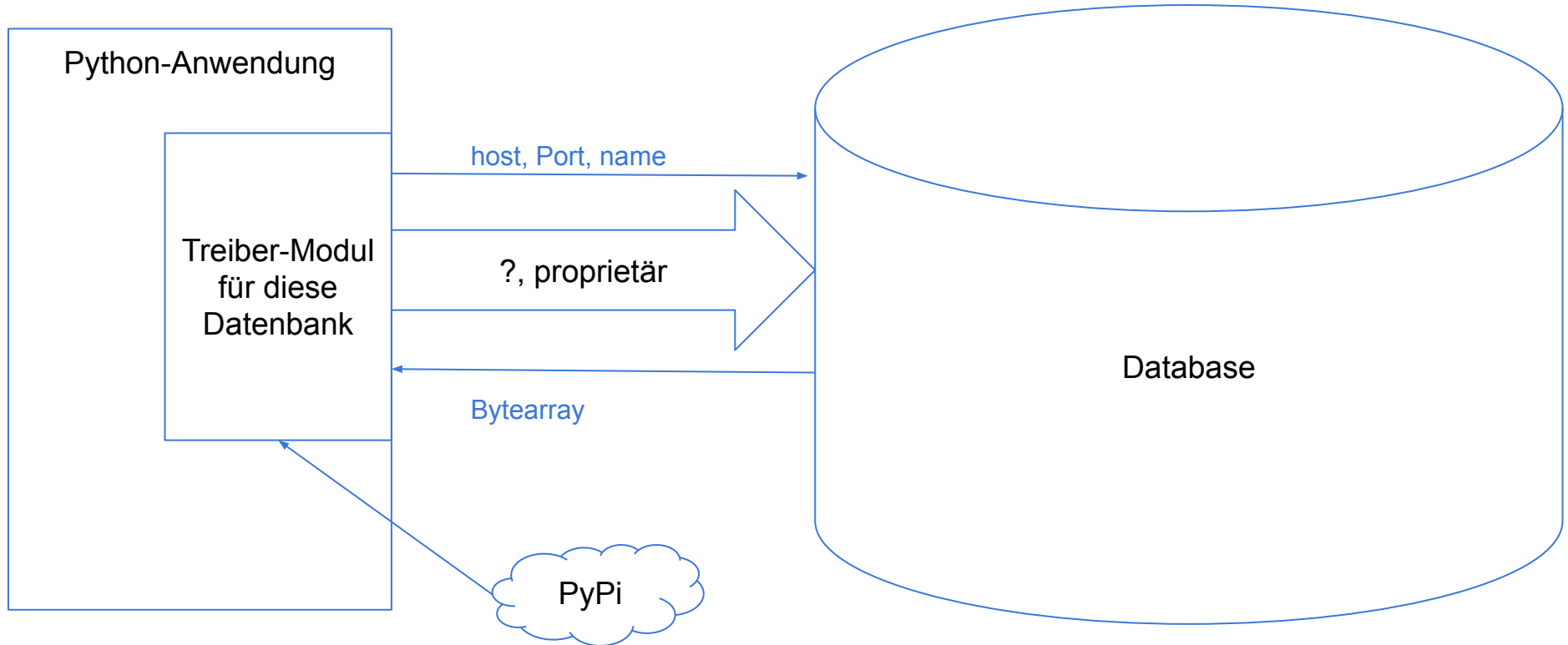
Python-Anwendung

requests
json

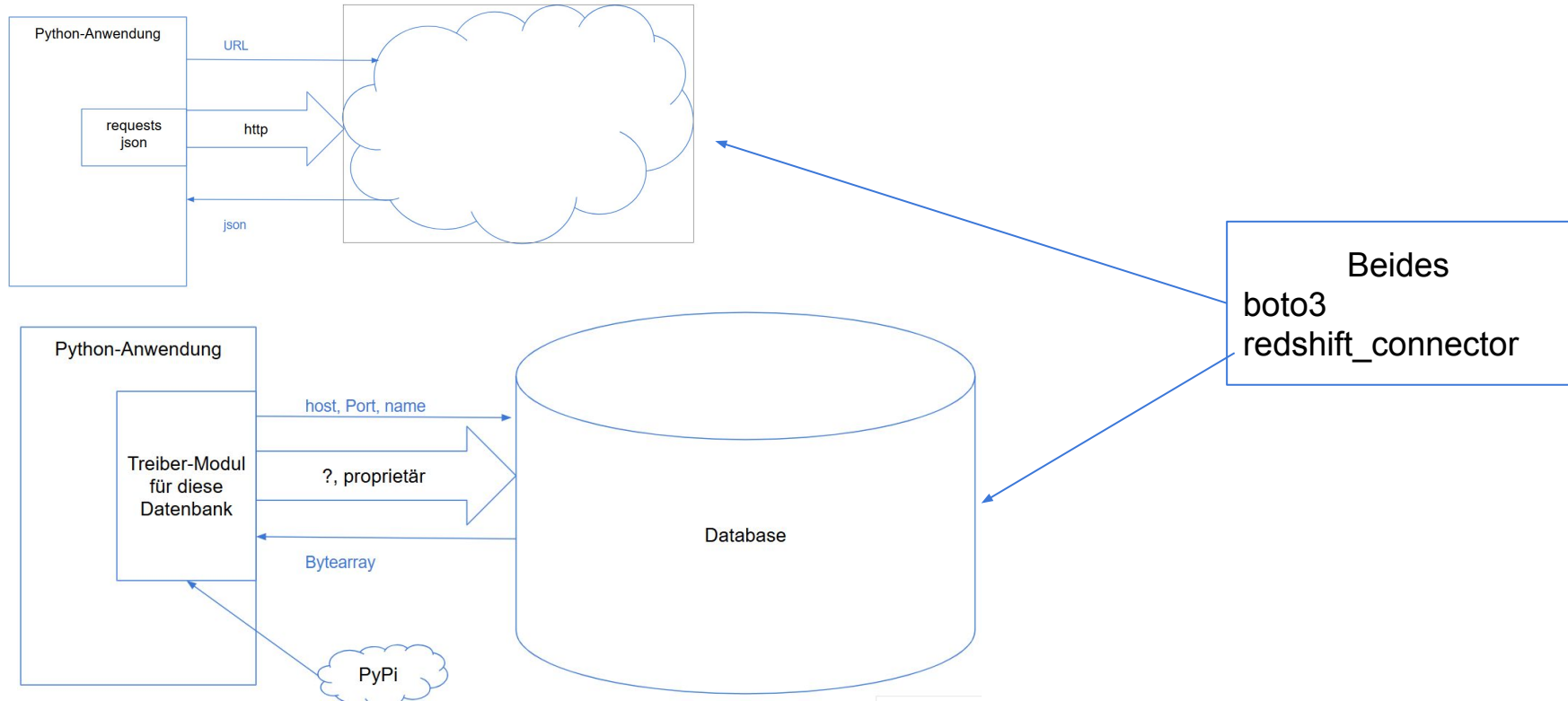
URL

http

json



Zugriff auf Redshift (Datenquelle der Amazon Cloud)



Beispiel: MySql-Datenbank

Python-Anwendung

Treiber-Modul

mysql-connector
-python

javacream.eu, 3406,
javacream

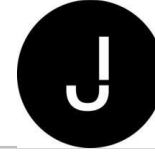
?, proprietär

Bytearray

PyPi

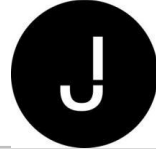
MySql

Tabellenschema
SHOPPING_LIST

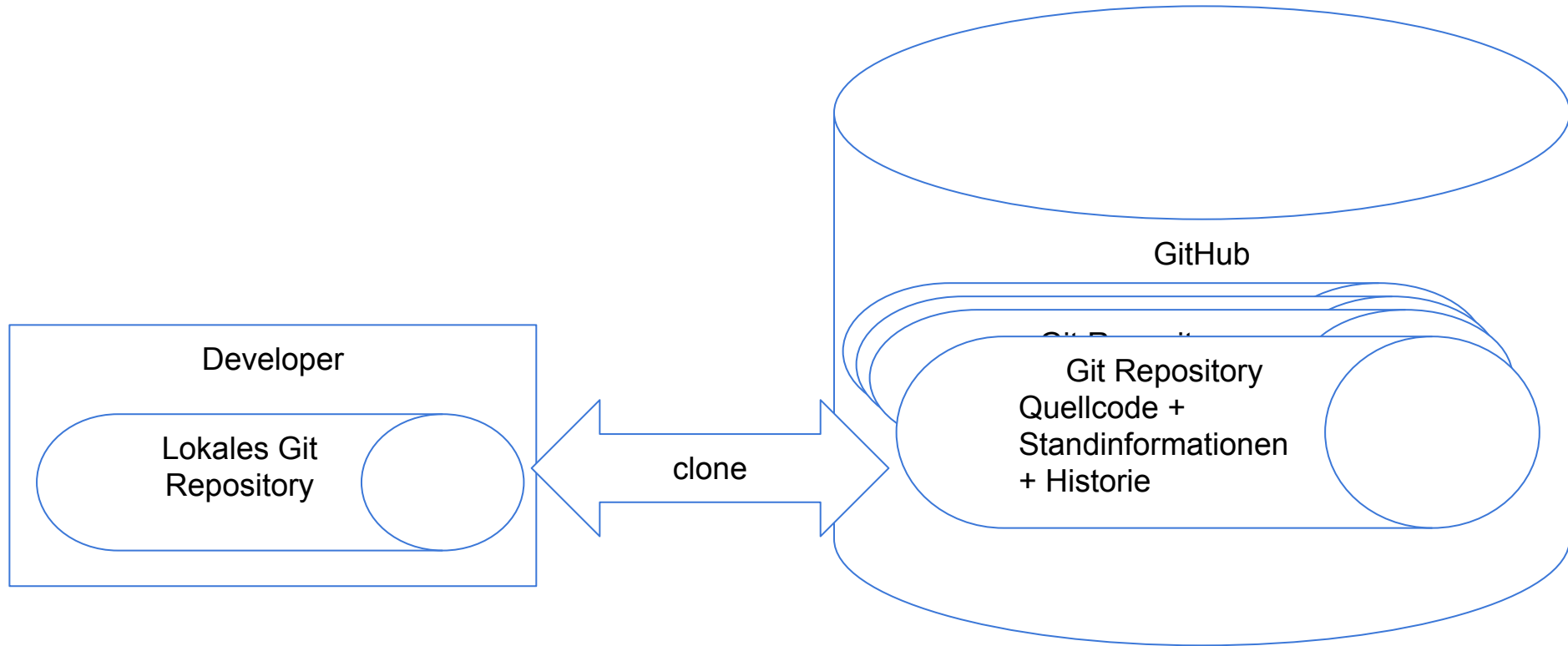


Exkurs Git

[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](#)



- Lokale Git-Installation
 - Check git --version
- Zugriff auf einen Git Server
 - Git Hub (meine Präsentation), GitLab, BitBucket, ...
 - Anmeldedaten sind hierfür erforderlich
 - Hinweis: Anmeldedaten für GitHub werden zur Verfügung gestellt
- Einmalige Vorarbeiten
 - git config --get user.name
 - git config --global user.name "Euer Name"
 - git config --global user.email eure@email.com
 -



Git Repository

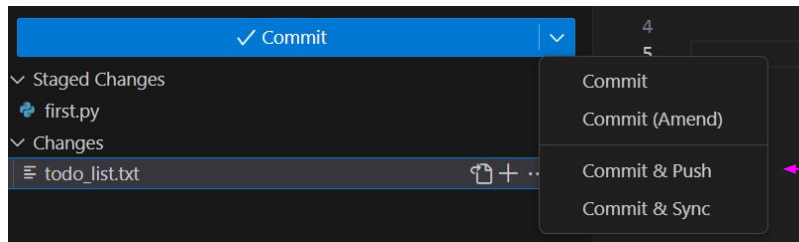
Quellcode +
Standinformationen +
Historie

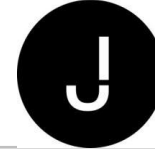
Quellcode +
Standinformationen +
Historie

Quellcode +
Standinformationen +
Historie

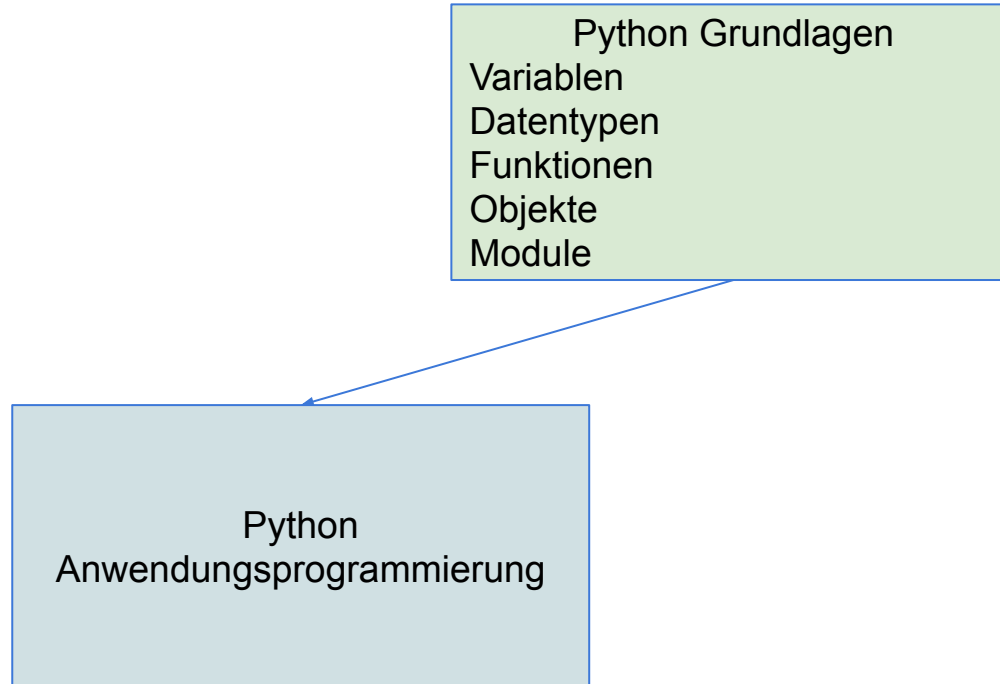
Branches, diese unterteilen das Repositories z.B. für einzelne User

- Normales Arbeiten, Programmieren, ...
- Falls ein "Stand" erreicht ist
 - Auswählen aller Dateien, die Änderungen im neuen Stand aufweisen
 - Eingabe eines beschreibenden Textes: Warum ist das ein neuer Stand
 - continue work -> OK, aber doch etwas mau
 - add first.py -> Besser
 - Klick "commit"
 - Menü - Push



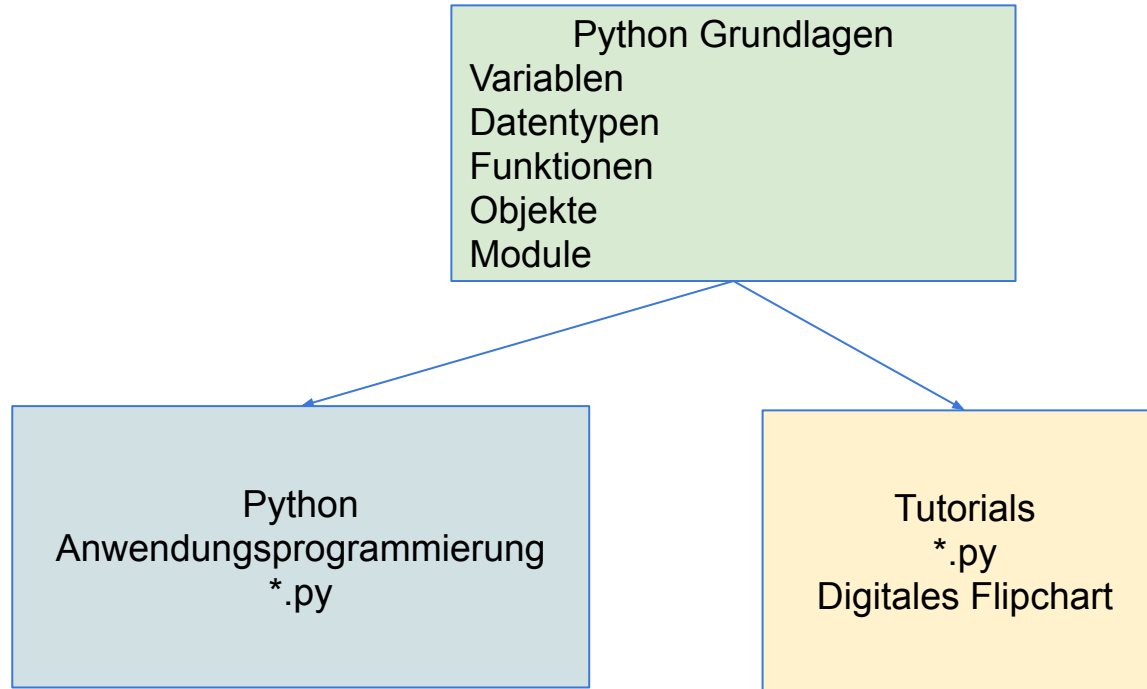


Jupyter



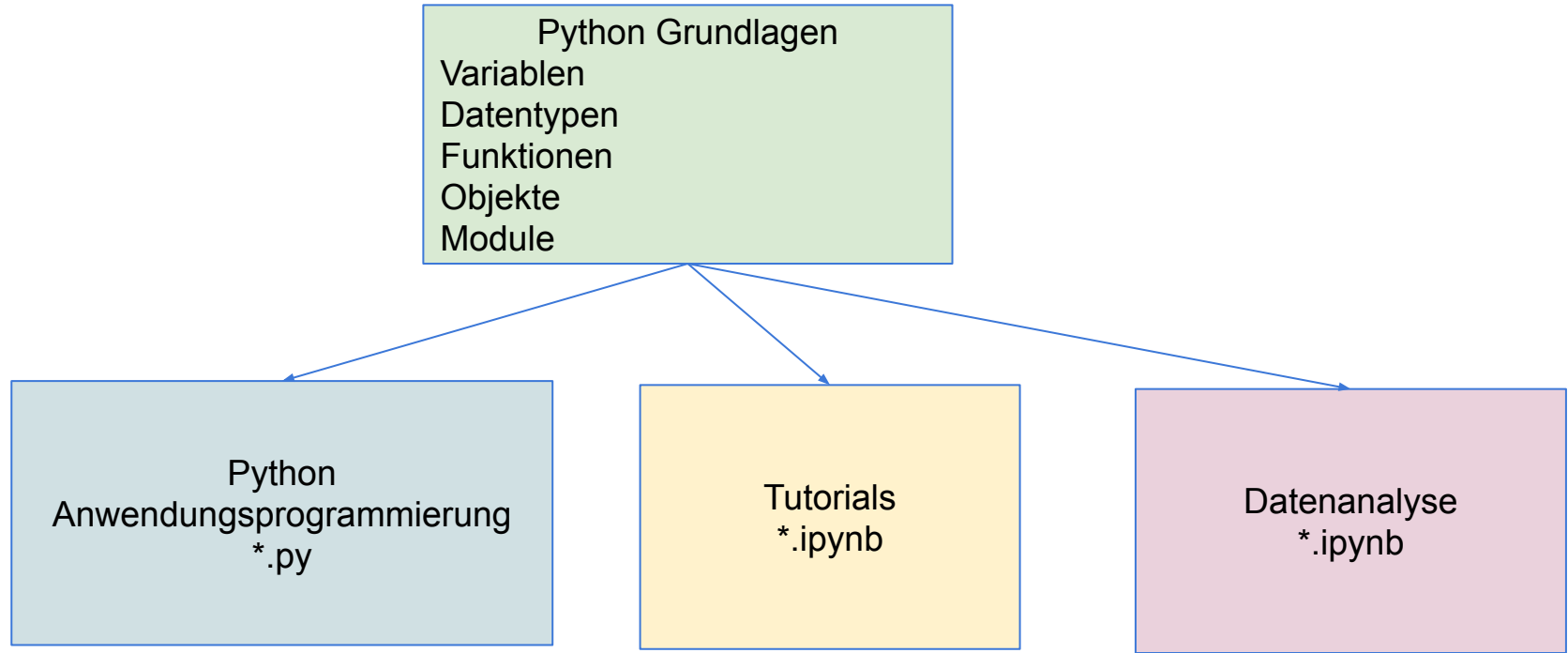
- Fachvorgabe
 - "Wir benötigen eine Anwendung, die einem User (oder anderen Systeme) bestimmte Features zur Verfügung stellt"
- Programmierung implementiert und testet diese Features in Python
 - Ergebnis ist ein Python Script
 - Definition of Done
 - Das Programm funktioniert so, wie es die Vorgabe spezifiziert hat
- Produktionsübergabe
 - Das Programm steht den AnwenderInnen zur Verfügung
 - Alternativ: Die Anwendung läuft scheduled im Hintergrund

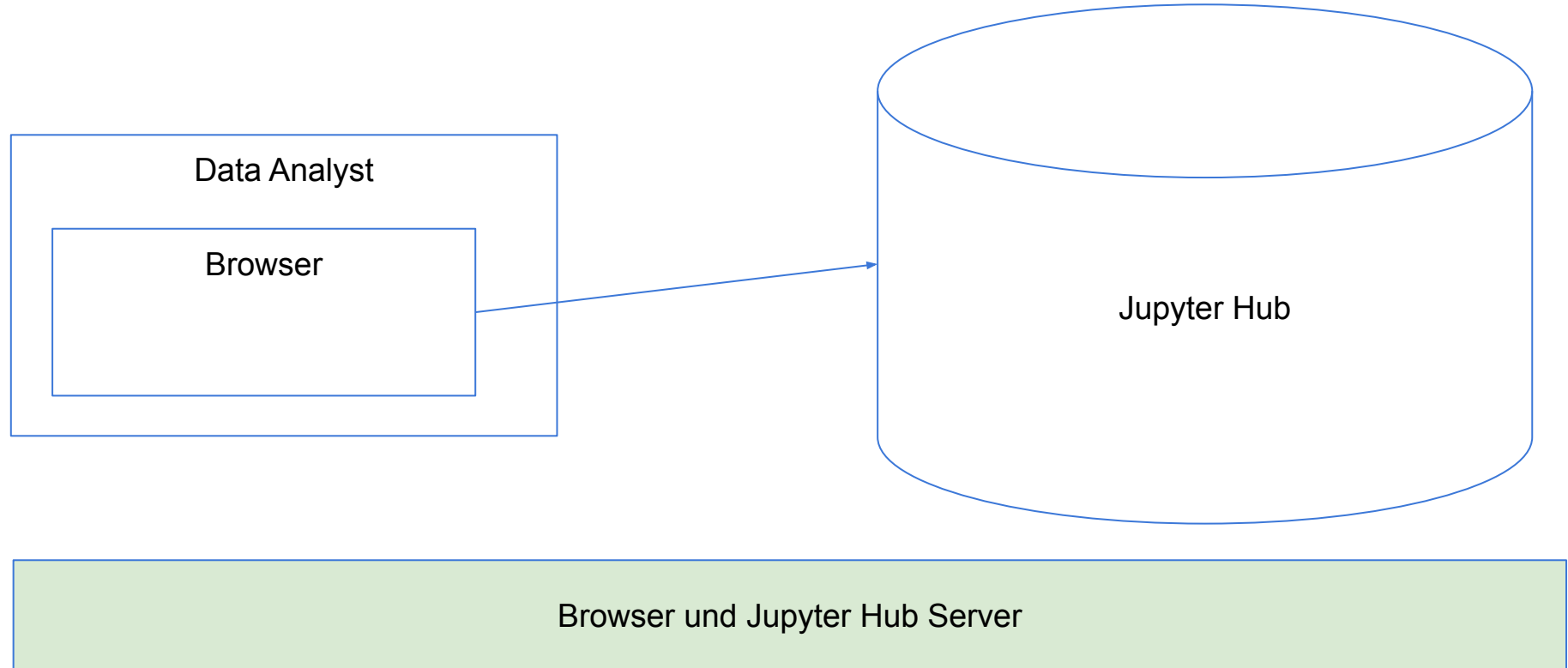
- Fachvorgabe
 - ~~Ich habe folgende Daten, bereite diese auf folgende Art und Weise auf und berechne Ergebnisse~~
 - "Ich habe Daten, schau Sie dir mal an..."
- Statt eines fixen Programms wird eine Datenanalyse verschiedene Dinge ausprobieren, durchspielen
 - Auch Irrwege sind durchaus wertvoll
- Präsentation und Interpretation der Ergebnisse ("Story Telling")
 - "Von der Korrelation zur Kausalität"
 - Definition of Done: Ein Publikum akzeptiert diese Interpretation und leitet daraus Maßnahmen ab



- Wünsche
 - Allgemein gültiges Format
 - Beschreibender / dokumentierender formatierter Freitext
 - Code-Bereiche
 - Code-Bereiche sollen in einer Runtime ausführbar sein
 - Ausführung der Codes soll auf einem Server stattfinden, Steuerung erfolgt bevorzugt im Browser
- Lösung
 - Jupyter
 - Name entstammt aus den 3 Programmiersprachen **Julia**, **Python** und **R**
 - Hinweis: Heute keine relevante Beschränkung auf diese Sprachen mehr
 - Java, C#, JavaScript, Shell, ...

- Jupyter Hub
 - Produkt, das Jupyter bereitstellt
 - Im Internet
 -
 - In der Cloud
 - Auf Unternehmensservern
 - Lokal
- Jupyter Notebooks
 - Ein Dokument, das aus beliebig vielen Zellen besteht
 - Freitextzellen im sogenannten Markdown-Format
 - Code-Zellen mit ausführbarem Code einer unterstützten Programmiersprache



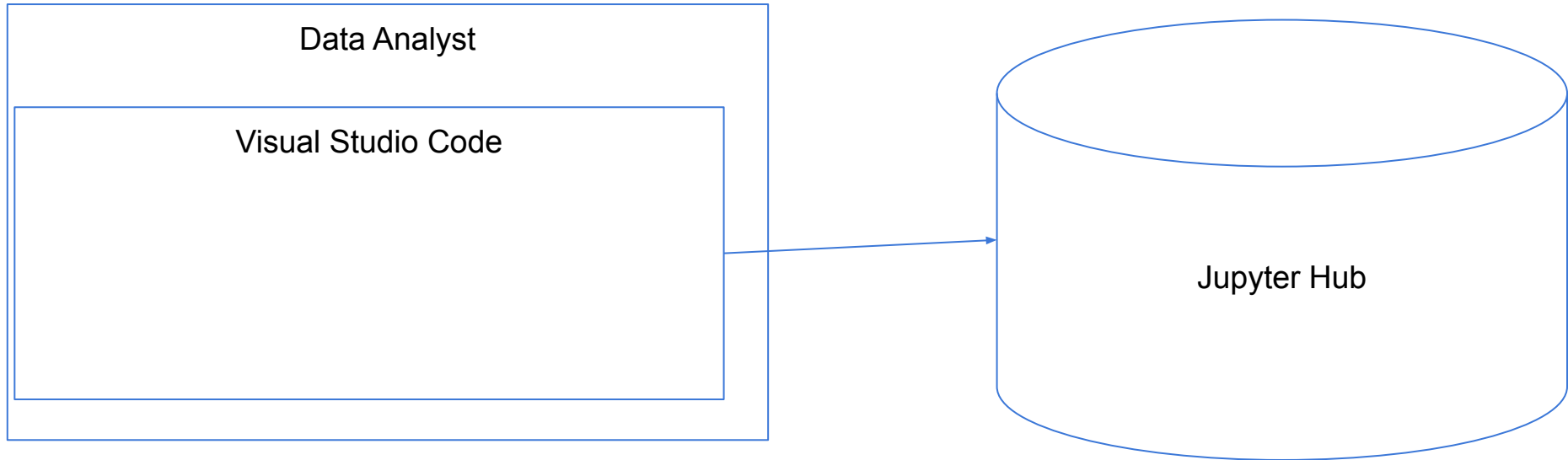


Data Analyst

Visual Studio Code

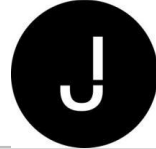
Jupyter Hub

Visual Studio Code und lokaler Jupyter Hub
(Unsere Trainingsumgebung)



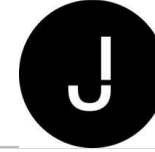
Visual Studio Code und Jupyter Hub Server
(Eine reale Arbeitsumgebung!)

- Markdown ist eine vereinfachte Auszeichnungssprache für formatierten Text
 - Sehr weit verbreitet
 - Beispiel: [README.md](#) in GitHub



- Legen Sie sich bitte auch ein Notebook an
- Machen Sie sich grob mit dem neuen Editor in Visual Studio Code vertraut!

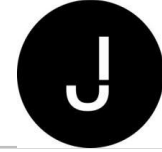
- Der Kernel des Notebooks
 - existiert pro Notebook
 - und ist Endeffekt eine Zellen-übergreifende Python-Runtime
 - Deklaration einer gelaufenen Zelle sind "überall" sichtbar
- Stilistische Änderung
 - Anwendungsprogrammierung nutzt Funktionen zur Codestrukturierung
 - Jupyter Notebooks nutzen die Zellen zur Strukturierung
 - Jupyter Notebooks müssen keine Funktionen deklarieren, auch kein main
- Code Assist, Syntax Highlighting, Warnungen / Fehler und auch der Debugger funktionieren wie gewohnt
- Falls die letzte Anweisung einer Zelle ein Ergebnis hat, wird dieses automatische ausgegeben
- Das Root-Verzeichnis des Notebooks ist das aktuelle Verzeichnis



Die Pandas-Bibliothek in Python



- -> `pandas_primer.ipynb`



- Beispiel: Kaggle
 - Hinweis
 - Kaggle verlangt eine freie Registrierung
 - Im Seminar nur "read-only"

- Bei Bedarf: Descriptive Statistic
- Diamonds-Beispiel um ein paar Statistik-Informationen bereichern
- Übersicht z.B. des Python-Tutoriums von Greg Hamel auf Kaggle
- Überblick Datensätze / Notebooks auf Kaggle

- Benutzereingabe eines Textes
- Das Programm soll die Anzahl der enthaltenen Vokale (a, e, i, o, u, A, E, I, O, U) bestimmen
 - Pseudocode
 - Eingabe des Textes
 - Iterieren über die Einzelzeichen
 - Einzelzeichen ein Vokal? -> Zähler hochzählen
 - Ausgabe: text enthält x Vokale